

UNIT IV – SEARCHING, SORTING, AND PROBLEM SOLVING TECHNIQUES

Searching: Binary search and variants , Sorting algorithms: Quick sort, Merge sort, Heap sort, Divide and conquer technique , Greedy algorithms – real-life examples (coin change, activity selection), Backtracking – N-Queens, Sudoku solver, Dynamic Programming: Knapsack, Longest Common Subsequence (LCS), Matrix Chain Multiplication ,Applications in: Inventory and resource optimization, pattern matching, scheduling

4.1 SEARCHING

Searching is an operation or a technique that helps finds the place of a given element.

Searching Techniques

To search an element in a given array, it can be done in following ways: 1. Sequential Search

2. Binary Search

Linear Search

- Sequential search is also called as Linear Search.
- Sequential search starts at the beginning of the list and checks every element of the list.
- It is a basic and simple search algorithm.
- Sequential search compares the element with all the other elements given in the list. If the element is matched, it returns the value index, else it returns -1.

A simple approach is to do linear search, i.e

- Start from the leftmost element of arr[] and one by one compare x with each element of arr[] If x matches with an element, return the index. **4.2 BINARY**

SEARCH AND VARIANTS

Definition: A binary search or half-interval search algorithm finds the position of a specified value within a sorted array.

- Search a sorted array by repeatedly dividing the search interval in half.
- Begin with an interval covering the whole array.
- If the value of the search key is less than the item in the middle of the

interval, narrow the interval to the lower half.

- Otherwise narrow it to the upper half.
- Repeatedly check until the value is found or the interval is empty.

If searching for 23 in the 10-element array:



Algorithm

- Compare x with the middle element
- If x matches with middle element, we return the mid index.
- Else If x is greater than the mid element, then x can only lie in right half subarray after the mid element. So we recur for right half.
- Else (x is smaller) recur for the left half.

How Binary Search Works?

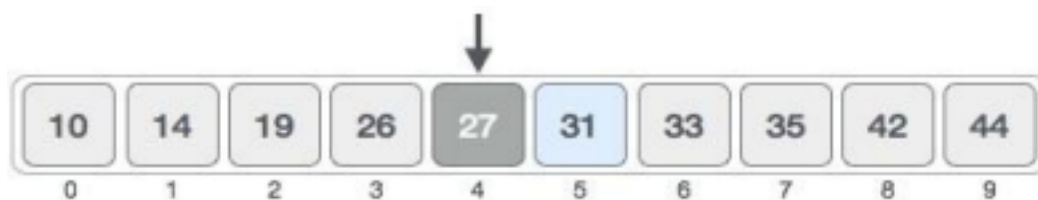
- For a binary search to work, it is mandatory for the target array to be sorted. •

The following is our sorted array and let us assume that we need to search the location of value 31 using binary search



First, we shall determine half of the array by using this formula – $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$

Here it is, $0 + (9 - 0) / 2 = 4$ (integer value of 4.5). So, 4 is the mid of the array.



Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value

must be in the upper portion of the array.



We change our low to mid + 1 and find the new mid value again. $\text{low} = \text{mid} + 1$

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2 = 5 + (9 - 5) / 2 = 5 + 2 = 7$$

Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.



The value stored at location 7 is not a match, rather it is more than what we are looking for. So, the value must be in the lower part from this location

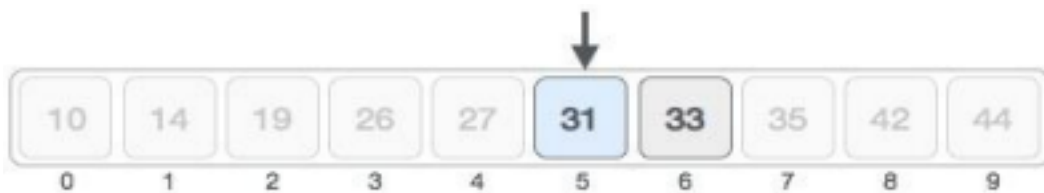


Hence, we calculate the mid again. This time it is 5

$$\text{high} = \text{mid} - 1 \quad \text{high} = 7 - 1 = 6$$

Low=5

$$\begin{aligned} \text{mid} &= \text{low} + (\text{high} - \text{low}) / 2 = 5 + (6 - 5) / 2 \\ &= 5 + 0.5 = 5.5 = 5 \end{aligned}$$



We compare the value stored at location 5 with our target value. We find that it is a match.



We conclude that the target value 31 is stored at location

5. Time Complexity

- Linear Search $O(n)$
- Binary Search $O(\log n)$

Program

```
def binary_search_recursive(arr, target, low, high):
    if low > high:
        return -1 # element not found
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, high)
    else:
        return binary_search_recursive(arr, target, low, mid - 1) #
Example usage
arr = [1, 3, 4, 6, 8, 10]
target = 4
result = binary_search_recursive(arr, target, 0, len(arr) - 1)
if result != -1:
    print(f"Element {target} found at index {result}")
else:
    print(f"Element {target} not found")
```

Variants

1. Standard Binary Search

- Finds whether an element exists in a sorted array.
- Returns any index of the target if found.
- Can be iterative or recursive.
- Time Complexity: $O(\log n)$

2. Lower Bound (Leftmost)

- Finds the first occurrence of a target (or the first element $\geq$ target).
- Useful when duplicates exist

3. Upper Bound (Rightmost)

- Finds the last occurrence of a target

4. First True / Predicate Binary Search

- Used when you are searching for the first index where a condition becomes true.

5. Last True / Predicate Binary Search

- Finds the last index where a condition is true.

4.3 SORTING ALGORITHMS

Divide and Conquer is a powerful algorithm design technique that breaks a problem into smaller subproblems, solves each subproblem independently, and then combines their results to get the final solution.

The Divide and Conquer technique is a powerful algorithm design paradigm that works in three steps:

Divide – Break the problem into smaller subproblems.

Conquer – Solve each subproblem recursively.

Combine – Merge the results of the subproblems to get the final result.

Examples

Algorithm	Key Operation	Combine Step
Merge Sort		Divide array in halves Merge two sorted halves
Quick Sort	Chooses pivot and divides	Concatenate sorted subarrays
Binary Search	Divides search interval	
Strassen’s Matrix Multiplication	Divides matrices	Combine smaller results

Advantages

- Efficient for large problems.
- Makes complex problems easier to understand and implement.
- Parallelism possible (subproblems can be solved independently).

Disadvantages

- Recursion overhead (function calls).
- May require additional memory (e.g., Merge Sort needs temporary arrays).

SORTING

Sorting refers to arranging data in a particular format. Sorting algorithm

specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order.

Examples for Sorting Algorithms

- Bubble Sort
- Radix Sort
- Insertion Sort
- Shell Sort
- Quick Sort etc

Sorting Algorithms

It is defined as an algorithm that puts the elements of a list in a certain order, which can be numerical order, alphabetical order, or any user-defined order. There are two types of sorting:

- Internal sorting- deals with sorting the data stored in the computer's main memory
- External sorting - deals with sorting the data stored in secondary memory. External sorting is applied when there is large data that cannot be stored in the main memory.

4.4 QUICK SORT

Quick Sort is a divide-and-conquer sorting algorithm widely used due to its efficiency and simplicity. It works by selecting a "pivot" element, partitioning the array into two subarrays (elements less than the pivot and elements greater than the pivot), and then recursively sorting the subarrays.

Key Steps of Quick Sort:

1. **Choose a Pivot:** Select an element as the pivot (commonly the first, last, or middle element).
2. **Partitioning:** Rearrange the array so that:
 - Elements smaller than the pivot are on the left.
 - Elements larger than the pivot are on the right.
3. **Recursive Sorting:** Apply the same process to the left and right subarrays.

Time Complexity:

- **Best/Average Case:** $O(n \log n)$

Code

```
def partition(arr, low, high):
```

```

pivot = arr[high] # Choosing the last element as pivot
i = low - 1 # Index of smaller element

for j in range(low, high):
    if arr[j] < pivot:
        i += 1
        swap(arr, i, j)
        swap(arr, i + 1, high)
    return i + 1

# swap function
def swap(arr, i, j):
    arr[i], arr[j] = arr[j], arr[i]

```

```

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high) # Partition index
        quick_sort(arr, low, pi - 1) # Sort left sub-array
        quick_sort(arr, pi + 1, high) # Sort right sub-array #

```

Example usage

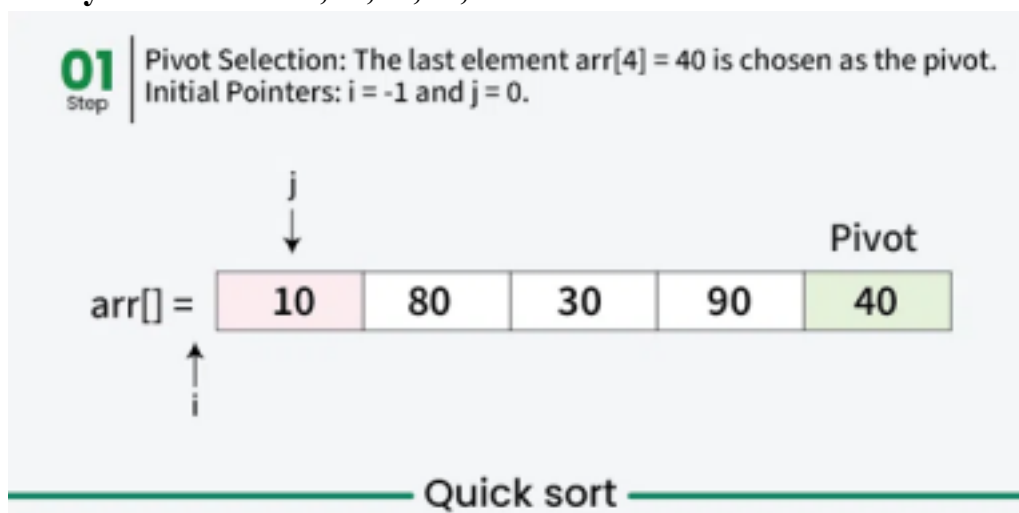
```

arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array:", arr)

```

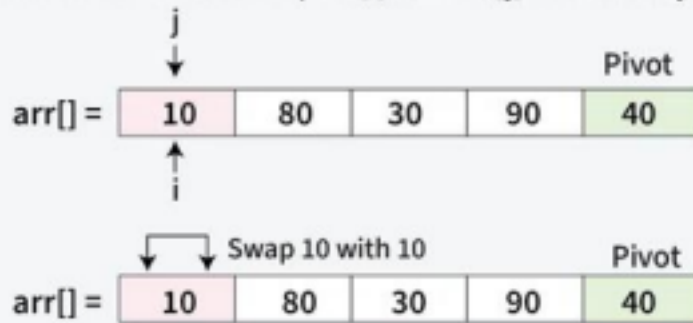
Example:

Array elements= 10,80,30,90,40



02
Step

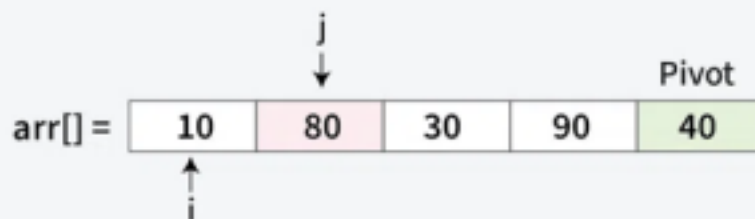
Since, $\text{arr}[j] < \text{pivot}$ ($10 < 40$)
Increment i to 0 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1



Quick sort

03
Step

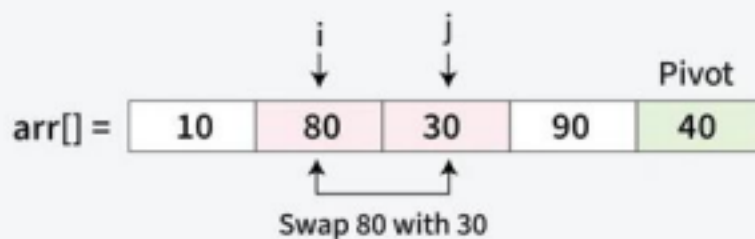
Since, $\text{arr}[j] > \text{pivot}$ ($80 > 40$)
No swap needed. Increment j by 1



Quick sort

04
Step

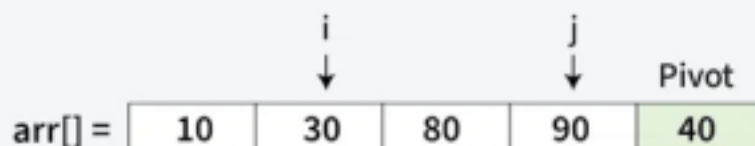
Since, $\text{arr}[j] < \text{pivot}$ ($30 < 40$)
Increment i by 1 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1



Quick sort

05
Step

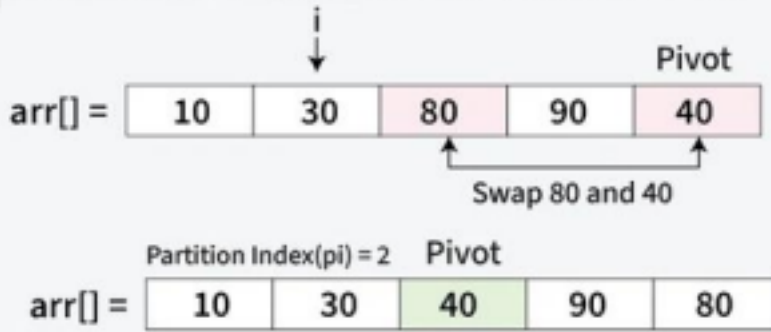
Since, $\text{arr}[j] > \text{pivot}$ ($90 > 40$)
No swap needed. Increment j by 1



Quick sort

06
step

Since traversal of j has ended. Now move pivot to its correct position, Swap $\text{arr}[i + 1] = \text{arr}[2]$ with $\text{arr}[4] = 40$.



Quick sort

In the previous step, we looked at how the partitioning process rearranges the array based on the chosen pivot. Next, we apply the same method recursively to the smaller sub-arrays on the left and right of the pivot. Each time, we select new pivots and partition the arrays again. This process continues until only one element is left, which is always sorted. Once every element is in its correct position, the entire array is sorted.

Advantages of Quick Sort

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.

Disadvantages of Quick Sort

- It has a worst-case time complexity of $O(n^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.

4.5 MERGE SORT

- Merge Sort is one of the most popular sorting algorithms that is based on the principle of Divide and Conquer Algorithm.
- Here, a problem is divided into multiple sub-problems. Each sub-problem is solved individually. Finally, sub-problems are combined to form the final solution.

Merge Sort Algorithm

- MergeSort is a recursive sorting procedure that uses at most $O(n \lg(n))$ comparisons.
- To sort an array of n elements, we perform the following steps in sequence:

- If $n < 2$ then the array is already sorted.
- Otherwise, $n > 1$, and we perform the following three steps in sequence:
 1. **Sort** the left half of the the array using MergeSort.
 2. **Sort** the right half of the the array using MergeSort.
 3. Merge the sorted left and right halves.

Algorithm

Step 1: Start

Step 2: Declare array and left, right, mid variable

Step 3: Perform merge

function. if left > right

return

mid= (left+right)/2

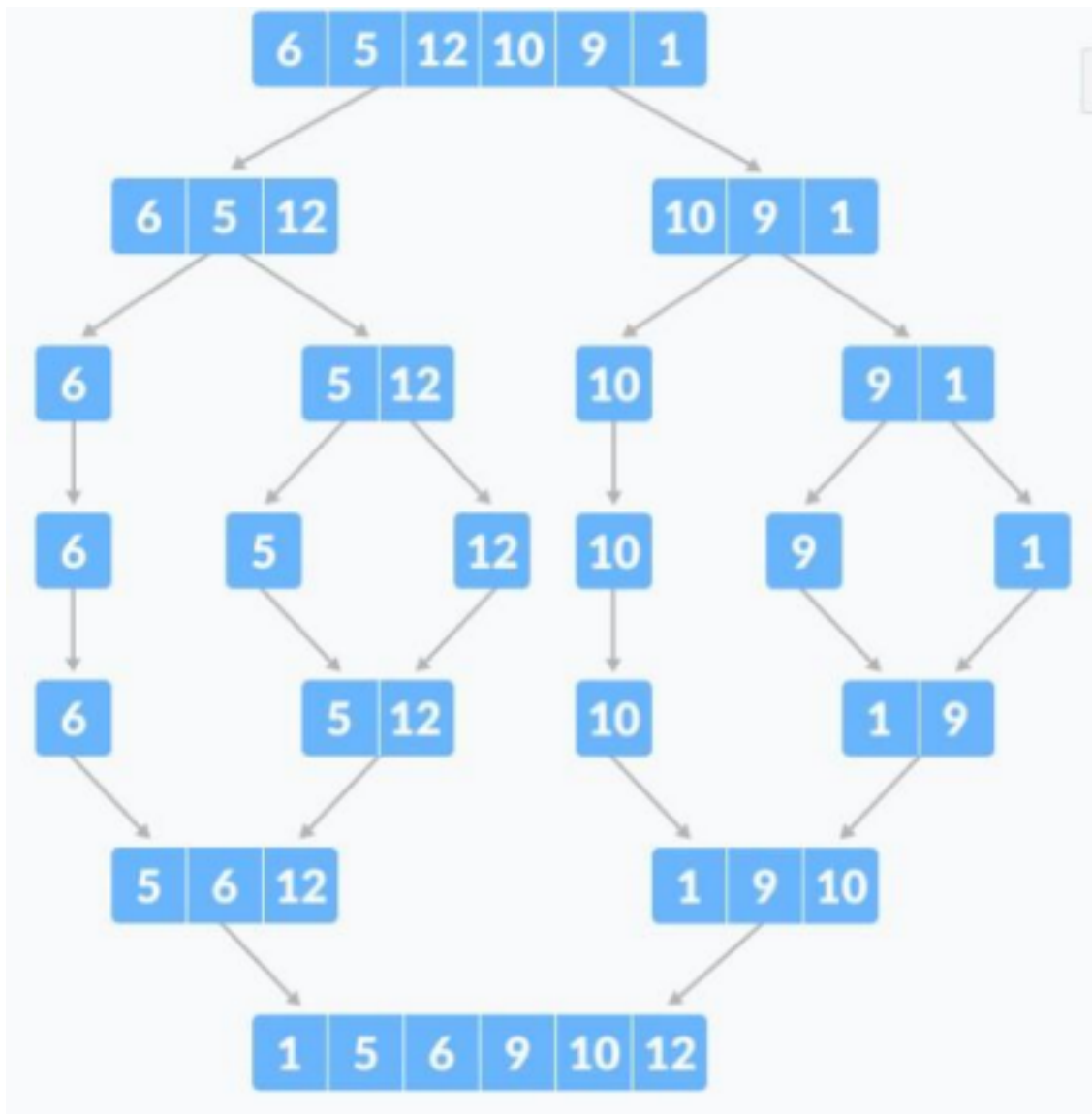
mergesort(array, left, mid)

mergesort(array, mid+1,

right) merge(array, left, mid,

right)

Step 4: Stop



How to Merge

Here are two lists to be merged:

First: (12, 16, 17, 20, 21, 27)

Second: (9, 10, 11, 12, 19)

Compare 12 and 9

First: (12, 16, 17, 20, 21, 27)

Second: (10, 11, 12, 19)

New: (9)

Compare 12 and 10

First: (12, 16, 17, 20, 21, 27)

Second: (11, 12, 19)

New: (9, 10)

Compare 12 and 11

First: (12, 16, 17, 20, 21, 27)

Second: (12, 19)

New: (9, 10, 11)

Compare 12 and 12

First: (16, 17, 20, 21, 27)

Second: (12, 19)

New: (9, 10, 11, 12)

Compare 16 and 12

First: (16, 17, 20, 21, 27)

Second: (19)

New: (9, 10, 11, 12, 12)

Compare 16 and 19

First: (17, 20, 21, 27)

Second: (19)

New: (9, 10, 11, 12, 12, 16)

Compare 17 and 19

First: (20, 21, 27)

Second: (19)

New: (9, 10, 11, 12, 12, 16, 17)

Compare 20 and 19

First: (20, 21, 27)

Second: ()

New: (9, 10, 11, 12, 12, 16, 17, 19)

Checkout 20 and empty list

First: ()

Second: ()

New: (9, 10, 11, 12, 12, 16, 17, 19, 20, 21, 27)

Program

```
def merge_sort(arr):
```

```
# Base case: if the array has one or zero elements, it's already sorted
```

```
if len(arr) <= 1:
```

```
    return arr
```

```
# Divide: Find the middle point to divide the array into two halves
```

```
mid = len(arr) // 2
```

```
# Recursively sort the left half and right half
```

```

left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])
# Combine: Merge the sorted halves
return merge(left, right)
def merge(left, right):
    sorted_array = []
    i = j = 0
    # Compare elements from both halves and merge them in sorted order
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_array.append(left[i])
            i += 1
        else:
            sorted_array.append(right[j])
            j += 1
    # If there are any remaining elements in the left half, add them
    while i < len(left):
        sorted_array.append(left[i])
        i += 1
    # If there are any remaining elements in the right half, add them
    while j < len(right):
        sorted_array.append(right[j])
        j += 1
    return sorted_array

```

4.6 HEAP SORT

- Heap sort is a comparison-based sorting technique based on Binary Heap Data Structure.
- In Heap Sort, we use Binary Heap so that we can quickly find and move the max element in $O(\log n)$ instead of $O(n)$

Phases

It operates in two main phases :

- **Building the Heap:**

The initial unsorted array is transformed into a binary heap. This can be either

a max-heap (where each parent node is greater than or equal to its children) or a min-heap (where each parent node is less than or equal to its children). • **Sorting the Elements**

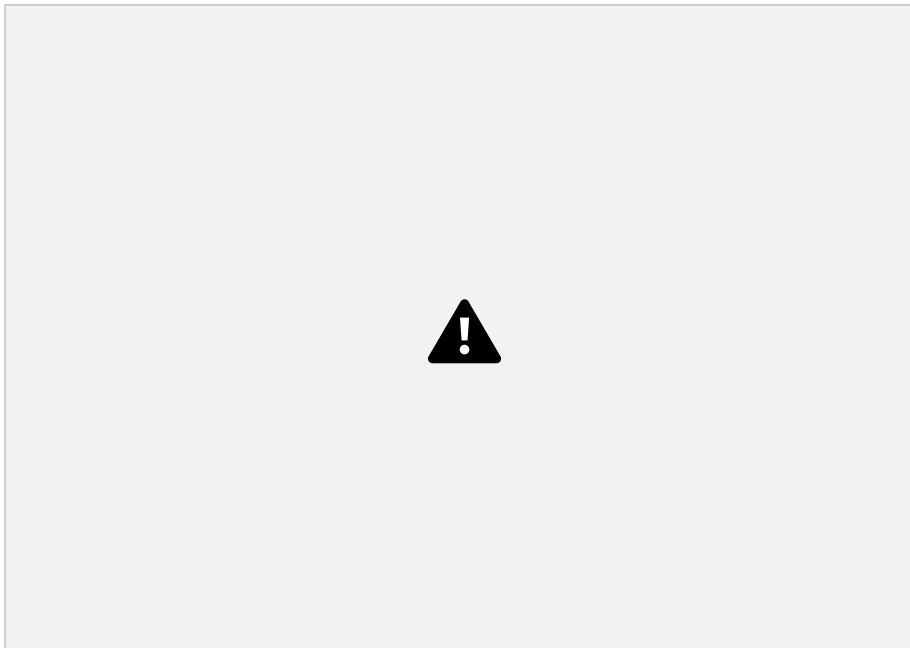
1. The largest (or smallest) element, which is at the root of the heap, is extracted and placed at the end of the array (or beginning, depending on ascending/descending order).
2. The heap is then restructured to maintain its heap property by “heapifying” the remaining elements. This typically involves moving the last element of the heap to the root and then sifting it down to its correct position.
3. This process of extracting the root and re-heapifying is repeated until all elements are in their sorted positions.

Example

- We have an array having elements `arr[] = [3, 7, 1, 8, 2, 5, 9, 4, 6]`.

Creating the Heap

- We are just inserting element from left to right in order to make a tree.



- Step 1. The Heapify Method is going to apply in order to make max or min heap , we are going to apply here max heap .





Phase 2: Sorting the Array

- In phase 2, we take advantage of the fact that the largest element of the max heap is always at its root (in the array: on the far left). this time we take a empty list or array and start deleting the node from the heap what we made in 1st phase .
- while deleting we keep the last node element from the heap and put at the deleted root place you can see below

```
def heapify(arr, n, i):
```

```
# Assume the current node (i) is the largest
```

```
largest = i
```

```
left = 2 * i + 1 # left child index
```

```
right = 2 * i + 2 # right child index
```

```
# Check if left child exists and is greater than root
```

```
if left < n and arr[left] > arr[largest]:
```

```
largest = left
```

```
# Check if right child exists and is greater than current largest
```

```
if right < n and arr[right] > arr[largest]:
```

```
largest = right
```

```
# If the largest is not the root, swap and continue heapifying
```

```
if largest != i:
```

```
arr[i], arr[largest] = arr[largest], arr[i]
```

```
heapify(arr, n, largest)
```

```
def heap_sort(arr):
```

```
    n = len(arr)
```

```
    # Step 1: Build a max heap
```

```
    for i in range(n // 2 - 1, -1, -1):
```

```
        heapify(arr, n, i)
```

```
    # Step 2: Extract elements one by one
```

```
    for i in range(n - 1, 0, -1):
```

```
        # Move current root to the end
```

```
        arr[i], arr[0] = arr[0], arr[i]
```

```
        # Heapify the reduced heap
```

```
        heapify(arr, i, 0)
```

```
    # Example usage
```

```
    arr = [12, 11, 13, 5, 6, 7]
```

```
    print("Original array:", arr)
```

```
    heap_sort(arr)
```

```
    print("Sorted array:", arr)
```

4.7 GREEDY ALGORITHMS

DEFINITION

- A greedy algorithm is a problem-solving technique that makes a sequence of choices by selecting the best possible option available at each step.
- The key characteristic of a greedy algorithm is that it makes these choices in a locally optimal way, with the hope that these local choices will lead to a globally optimal solution.

Advantages

- Unlike more complex algorithms that use backtracking or dynamic

[programming](#), greedy algorithms do not revisit past decisions or adjust their approach based on future possibilities.

- Once a choice is made, it is never reconsidered.
- This makes greedy algorithms **fast and efficient**, often running in **linear or logarithmic time**, depending on the problem.

Disadvantages

- However, this same property also means the greedy algorithm does not always guarantee the best possible solution in every case.

Optimization problems

- Given a problem instance, a set of constraints and an objective function.
- Find a feasible solution for the given instance for which the objective function has an optimal value
- Either maximum or minimum depending on the problem being solved.
- A feasible solution satisfies the problem's constraints
- The constraints specify the limitations on the required solutions.

Working

To understand how a greedy algorithm works, let's break it down into simple steps:

- **Make a choice** – At each step, pick the best available option based on a specific criterion.
- **Proceed to the next step** – Move forward and repeat the process until the problem is solved.
- **Check the final outcome** – The algorithm arrives at a solution that is either optimal or close to optimal.

Problems solved with greedy algorithm

- Coin change problem
- Fractional knapsack
- Activity Selection
- 0/1 Knapsack
- Dijkstra's shortest path algorithm
- Huffman Encoding
- MST (Prim's & Kruskal's Algorithm)

4.8 REAL LIFE EXAMPLES

Coin change problem

- The Coin Change problem is a classic example of a greedy algorithm in action.
The task is: given a set of coin denominations, determine the minimum number of coins required to make a specific amount.

Greedy Approach:

- To solve this, the greedy algorithm picks the largest coin denomination first and reduces the target amount by that coin's value.
- This process repeats until the entire amount is made up using the least number of coins possible.

Example

- Imagine we have coin denominations of {1, 5, 10, 25}, and we need to make 30.
- Start with the largest coin (25). Select one 25-Rs coin.
- Remaining amount: $30 - 25 = 5$
- Next, choose the largest coin less than or equal to the remaining amount (5).
Select one 5-Rs coin.
- Remaining amount: $5 - 5 = 0$
- In this case, the greedy algorithm successfully uses just two coins (one 25-Rs coin and one 5-Rs coin) to make 30

Python Code

```
def coin_change_min(coins, amount):
    # Initialize dp array with "infinity"
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0 # Base case: 0 coins to make amount 0
    for coin in coins:
        for x in range(coin, amount + 1):
            dp[x] = min(dp[x], dp[x - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

# Example
coins = [1, 2, 5]
amount = 11

print("Minimum coins:", coin_change_min(coins, amount))
# Output: 3 (11 = 5 + 5 + 1)
```

Activity selection problem

- The **activity selection problem** is an example of a greedy algorithm where the maximum number of non-overlapping activities are selected from the given activity set. A person can complete one activity at a time. The activities are given in the form of their starting and completion times.
- We are given **n activities**, each with:
 - a **start time**
 - a **finish time**
- We need to select the **maximum number of activities** that can be performed by a single person, assuming **one activity at a time**.

Key Idea of the Greedy Technique

- The greedy strategy here is:

Always pick the activity that finishes earliest (among the remaining ones).

Why?

- If we choose the activity that finishes earliest, it leaves the most room for other activities to fit afterward.
- This ensures the maximum number of non-overlapping activities can be chosen.

Algorithm Steps

- Sort all activities by their finish times (earliest finishing activity comes first).
- Select the first activity (since it finishes earliest).
- For each next activity:
 - If its start time $\geq$ finish time of the last selected activity, then select it.
 - Otherwise, skip it.
- Continue until all activities are checked.

Example

Input:

arr[] = {{5,9},{1,2},{3,4},{0,6},{5,7},{8,9}}

Sorted Activities: {1,2}, {3,4},{0,6},{5,7},{5,9},{8,9}

Select First activity {1,2}

Continue selecting

- Next, **A2 (3–4)** → valid (starts after 2)
- Next, **A4 (5–7)** → valid (starts after 4)

- Next, **A5 (8–9)** → valid (starts after 7)
- Skip **A6 (5–9)** and **A3 (0–6)** because they overlap

Final Selected Set: (A1: 1–2), (A2: 3–4), (A4: 5–7), (A5: 8–9)

Time Complexity

- **Overall: $O(n \log n)$**

4.9 BACKTRACKING

- Backtracking is a problem-solving technique used for finding solutions by exploring all possible options, but systematically pruning (discarding) those that cannot lead to a valid solution.

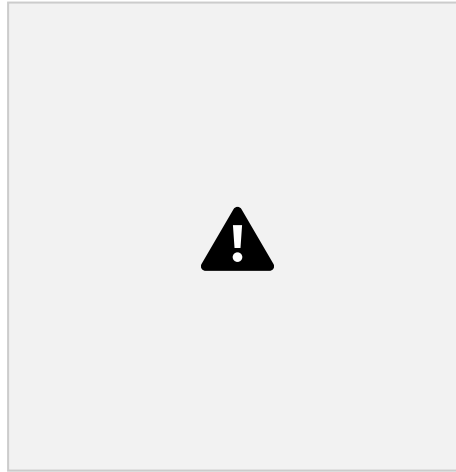
It works by:

- Building a solution step by step.
- Checking constraints at each step.
- If the current step leads to a dead end, the algorithm unwraps (backtracks) the last step and tries another option.
- Works on the principle of **Depth-First Search (DFS)**.
- Reduces the search space by eliminating invalid possibilities early.

Commonly applied in:

- **N-Queens Problem**
- **Sudoku Solver**
- **Maze Solving**
- **Graph Coloring**
- **Subset/Permutation generation**
- It is commonly used in situations where we need to explore multiple possibilities to solve a problem, like searching for a path in a maze or solving puzzles like Sudoku.
- When a dead end is reached, the algorithm backtracks to the previous decision point and explores a different path until a solution is found or all possibilities have been exhausted.
- We use backtracking when we need to explore all possible paths (or permutations). Instead of going through every path, we backtrack to avoid unnecessary work whenever we are sure that no path from here would lead to a valid solution.

Back tracking



Types of Backtracking Problems

Problems associated with backtracking can be categorized into 3

categories:

- Decision Problems: Here, we search for a feasible solution.

- Optimization Problems: For this type, we search for the best solution.

- Enumeration Problems: We find set of all possible feasible solutions to the problems of this type.

N-Queens Problem

- The N-Queens problem is the challenge of placing N queens on an $N \times N$ chessboard in such a way that no two queens attack each other.
- A queen can move any number of squares horizontally, vertically, or diagonally.
- Therefore, the conditions are:
 - No two queens in the same row
 - No two queens in the same column
 - No two queens in the same diagonal
- The goal is to find one or all valid arrangements of queens on the chessboard.

Time Complexity

- **$O(n!)$ For generating all permutations.**

Example

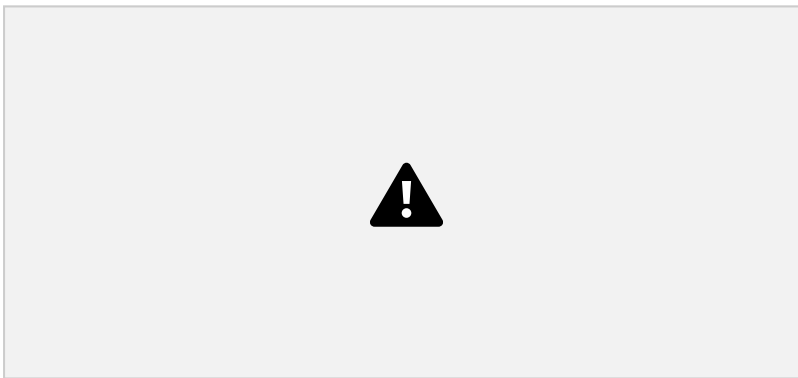
- **4 Queens Problem**

Algorithm

1. Start with an empty chessboard.
2. Place a queen in the first row (try each column one by one).
3. For the next row:
 1. Place a queen in a column such that it is safe (no conflict with already placed queens).

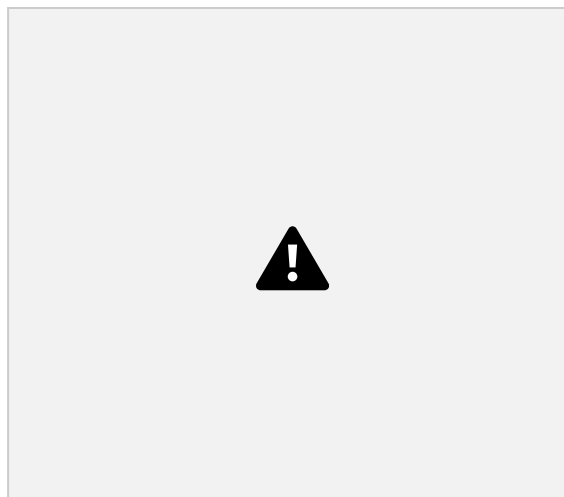
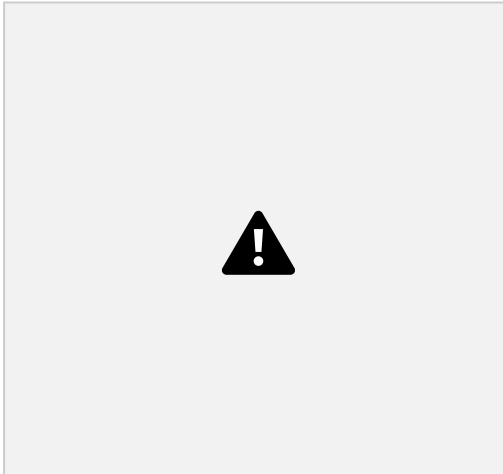
2. If safe → place queen and move to the next row.
3. If not safe → try the next column.
4. If no column is valid in the current row → backtrack (remove the queen from the previous row and try another position).
5. Repeat until:
 1. All queens are placed → solution found.
 2. All possibilities are exhausted → no solution exists.

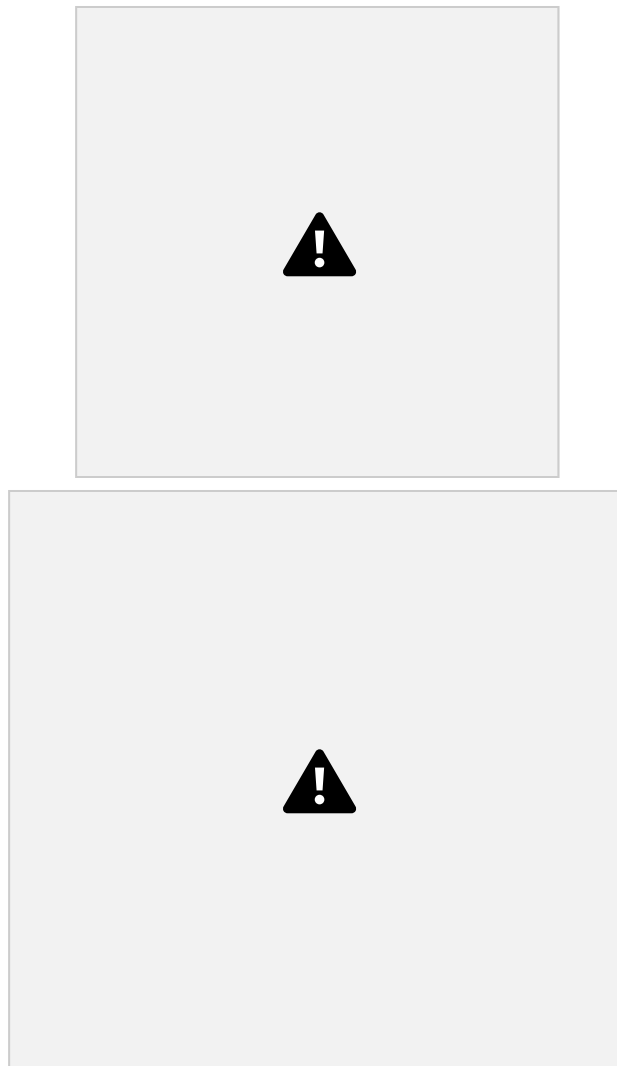
Solution



Step 1: Start with empty 4×4 board.

Step 2: Place queen in row 0 → column 1.





Sudoku puzzle

- A Sudoku puzzle is a 9×9 grid divided into 9 subgrids (3×3 blocks).
- The task is to fill the grid so that:
 - Each row contains digits 1–9 exactly once.
 - Each column contains digits 1–9 exactly once.
 - Each 3×3 subgrid contains digits 1–9 exactly once.
- Some cells are pre-filled, and empty cells must be filled following these rules.

Why Backtracking?

- Sudoku requires searching for valid numbers in empty cells.
- At each step, we must decide which number fits without breaking rules. • If a number leads to a dead end later, we must undo (backtrack) and try another option.

Backtracking Algorithm for Sudoku

- Find an empty cell (with value 0).
- Try placing numbers 1–9 in that cell.
- For each number:
 - Check if it is safe (not already present in the row, column, or 3×3

subgrid).

- If safe, place the number and move to the next empty cell (recursive call).
- If at some point no valid number can be placed:
 - Backtrack → reset the cell to 0 and return to the previous step to try a different number.
- Continue until:
 - The board is completely filled → solution found.
 - All options are exhausted → no solution exists.

Time Complexity:

- Worst-case ~ $O(9^n)$, where n is the number of empty cells.





4.10 APPLICATIONS: INVENTORY AND RESOURCE OPTIMIZATION, PATTERN MATCHING, SCHEDULING

- Problem-solving techniques such as **greedy algorithms, dynamic programming, backtracking, linear programming, and heuristics** are widely used in:
 - ❖ Optimizing inventory/resources,
 - ❖ Identifying patterns efficiently,
 - ❖ Scheduling tasks and resources effectively.

1) Inventory and Resource Optimization

Objective

Efficiently manage inventory and resources to **minimize costs** and **maximize utilization** while meeting demand.

Inventory Optimization → Ensuring the right quantity of stock (not too much, not too little).

Resource Optimization → Efficiently allocating limited resources (time, space, workers, machines, etc.) to maximize performance or minimize cost. **What We're Optimizing:**

- Inventory: Items, goods, or data to store, retrieve
- Resources: CPU time, memory, disk, bandwidth, labor, budget, etc. •

Optimization Goal: Maximize efficiency, minimize cost/waste, meet demand/service levels

Problem-Solving Techniques Used

Technique	How it’s Used in Inventory/Resource Optimization
Dynamic Programming	Optimize multi-stage inventory decisions (e.g., multi period stock planning).
Greedy Algorithms	Make locally optimal choices, e.g., prioritize high demand items for stocking.

(A) Greedy Techniques

- Use Case: Allocate resources step by step, always choosing the best option at each stage.

Examples in Inventory:

- Warehouse space allocation – Assign items to nearest free shelf (using priority queues / heaps).
- Truck loading – Choose items with highest value/weight ratio first (like Fractional Knapsack).

Data Structures Used: Heap, Priority Queue, Arrays, Graphs.

(B) Dynamic Programming (DP)

- Use Case: Optimize inventory costs with constraints (multi-period, multiple suppliers).

Examples:

- Minimize stock-out costs → Decide how much to reorder each month (like DP-based Knapsack).

- Resource scheduling → Assign machines to jobs minimizing idle time.

Data Structures: 2D arrays/tables, hash maps.

Problem Examples

1. Knapsack Problem (Dynamic Programming or Greedy)

Use case: Decide which inventory items to store/ship within weight or cost limits. • Goal: Maximize value under resource constraints.

- Variants: 0/1 Knapsack, Fractional Knapsack, Multi-dimensional Knapsack

2. Bin Packing Problem (Greedy / Backtracking)

Use case: Optimally pack items (inventory) into limited containers (resources). • Goal: Minimize number of bins used.

- Real-world: Cloud resource allocation, logistics container loading

II) Pattern Matching

Pattern matching means finding a substring (pattern) inside a larger text (string). Example:

Text: `"ABABDABACDABABCABAB"``

Pattern: `"ABABCABAB"``

We need to check if the pattern occurs in the text, and where?

Applications:

Searching keywords, DNA sequence matching, plagiarism detection, spam filtering, intrusion detection, etc.

Problem-Solving Techniques for Pattern Matching

1. Naïve Approach (Brute Force)

- The Naïve Pattern Matching algorithm is the simplest method of finding a pattern in a text.
- It tries to match the pattern with the text starting from each position, one by one.
- If the pattern matches → we record that position.
- If not → shift the pattern by one position and try again.

Working Example

Text (T): `ABABABC` (length = 7)

Pattern (P): `ABC` (length = 3)

Step-by-step:

1. Compare `ABC` with `ABA` → mismatch.
2. Shift by 1 → Compare `ABC` with `BAB` → mismatch.
3. Shift by 1 → Compare `ABC` with `ABA` → mismatch.
4. Shift by 1 → Compare `ABC` with `BAB` → mismatch.
5. Shift by 1 → Compare `ABC` with `ABC` → Match found at index 4.

Time Complexity

- Best Case: $O(n)$ → when pattern found quickly or mismatch occurs early.
- Overall $O(nm)$.

Why Efficient Algorithms?

- The Naïve (brute force) method takes $O(n \cdot m)$ time (n = text length, m = pattern length), which is inefficient for large data.
- Efficient algorithms use data structures (arrays, hashing, automata, tries, graphs) and problem-solving techniques to reduce repeated comparisons.

Algorithms

- KMP Algorithm (Knuth–Morris–Pratt): Uses preprocessing of the pattern to skip unnecessary comparisons.
- Rabin-Karp Algorithm: Uses hashing to match patterns efficiently.

Finite Automata Method: Builds an automaton to scan in $O(n)$ time. **III)**

What is Scheduling?

- Scheduling means allocating limited resources (CPU, tasks, machines, jobs, memory, workers, etc.) to competing tasks efficiently.
- In Operating Systems → scheduling CPU processes.
- In Real-world → job allocation, meeting scheduling, airline seat assignment, etc.
- Efficient scheduling requires problem-solving techniques and the right data structures.

Problem Solving Techniques for Scheduling

A) Greedy Technique

Idea: Always make the locally best decision.

Examples:

1. Interval Scheduling

- Interval Scheduling is a classic greedy algorithm problem:

- Given a set of jobs (tasks), each with a start time and an end time, we want to select the maximum number of jobs such that no two jobs overlap.
- It is also known as the Activity Selection Problem.

Real-Life Applications

- Meeting Room Scheduling (assign max non-overlapping meetings).
- Airline Runway Scheduling (maximize flights without conflict).
- Job Scheduling in OS (maximize CPU utilization).
- Sports Tournament Scheduling (avoid overlapping games).

Job Sequencing with Deadlines (maximize profit by scheduling jobs before deadlines).

- Job Sequencing with Deadlines is a Greedy scheduling problem where:
- We have `n` jobs. Each job has:
 - Deadline (d) → the latest time slot it must be finished.
 - Profit (p) → reward if job is completed before or at its deadline.
- Goal: Schedule jobs to maximize total profit, ensuring only one job per time slot and jobs complete before their deadline.
- Key Rule: If two jobs overlap, choose the one with higher profit.

Algorithm (Greedy Approach)

1. Sort all jobs by profit (descending order).
2. Create a time slot array (size = max deadline).
3. For each job (in sorted order):

Find the latest available slot $\leq$ job's deadline.

If free, assign the job there.

4. Return the scheduled jobs + maximum profit.

Example

| Job | Deadline | Profit |

| --- | ----- | ----- |

| J1 | 2 | 100 |

| J2 | 1 | 50 |

| J3 | 2 | 10 |

| J4 | 1 | 20 |

| J5 | 3 | 30 |

| J6 | 2 | 60 |

Step 1: Order: J1(100), J6(60), J2(50), J5(30), J4(20),

J3(10) Step 2: Determine number of slots

Slot 1 Slot 2 Slot 3

- - -

Step 3: Place Jobs in Latest available Slot $\leq$ Deadline

$3 \leq 2$ Cant take slot3

$2 \leq 2$ can use slot2

Slot 1 Slot 2 Slot 3

- J1 -

Next, J6 Deadline 2

Latest available slot $3 \leq 2$ False

slot 2 already occupied

slot $1 \leq 2$ So use Slot 1

Slot 1 Slot 2 Slot 3

J6 J1 -

Next J2 deadline 1

Latest slot available $3 \leq 1$ So reject

Next J5 deadline 3

Latest slot available $3 \leq 3$ So can use Slot 3

Slot 1 Slot 2 Slot 3

J6 J1 J5

Other jobs no free slots

Total Profit $100+60+30 = 190$

Scheduled Jobs J6 $\rightarrow$ J1 $\rightarrow$ J5

4.11 DYNAMIC PROGRAMMING

Definition

❖ Dynamic Programming is a method for designing algorithms. ❖ An algorithm

designed with Dynamic Programming divides the problem into subproblems, finds solutions to the subproblems, and puts them together to form a complete solution to the problem we want to solve.

- ❖ But unlike divide and conquer, these sub-problems are not solved independently. Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.
- ❖ Mostly, dynamic programming algorithms are used for solving optimization problems.

How it works???

- ❖ The problem should be able to be divided into smaller overlapping sub problem.
- ❖ Final optimum solution can be achieved by using an optimum solution of smaller sub-problems.
- ❖ Dynamic algorithms use memorization.

Some popular problems solved using Dynamic Programming

are ❖ [Fibonacci Numbers](#),

❖ Diff Utility ([Longest Common Subsequence](#)),

❖ [Bellman–Ford Shortest Path](#),

❖ [Floyd Warshall](#),

❖ [Edit Distance](#)

❖ [Matrix Chain Multiplication](#).

0/1 Knapsack Problem

- Given n items where each item has some weight and profit associated with it and also given a bag with capacity W , [i.e., the bag can hold at most W weight in it].
- The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.
- **Note:** The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Example

Find the optimal solution for the 0/1 knapsack problem making use of dynamic programming approach. Consider

$n = 4$

$w = 5 \text{ kg}$

$(w_1, w_2, w_3, w_4) = (2, 3, 4, 5)$

$(b_1, b_2, b_3, b_4) = (3, 4, 5, 6)$

A 2D array dp of size $(n+1) \times (W+1)$ is created, where $dp[i][j]$ represents the maximum benefit achievable with the first i items and a knapsack capacity of j .

Initialization:

The first row and first column of the dp table are initialized to 0, as no benefit is achieved with no items or no capacity.

Filling the Table:

- For each item i from 1 to n and each weight capacity j from 1 to W :
 - If the weight of the current item ($weights[i-1]$) is less than or equal to the current capacity j :
 - The decision is whether to include the current item or not.

$dp[i][j] = \max(dp[i-1][j], benefits[i-1] + dp[i-1][j - weights[i-1]])$ • This means the maximum benefit is either the benefit without including the current item ($dp[i-1][j]$) or the benefit of including the current item (its benefit plus the maximum benefit from the remaining capacity and previous items). • If the weight of the current item ($weights[i-1]$) is greater than the current capacity j :

The current item cannot be included.

$$dp[i][j] = dp[i-1][j]$$

Optimal Solution:

The optimal solution is found at $dp[n][W]$.

The optimal solution, the maximum benefit achievable with a knapsack capacity of 5 using 4 items, is $dp[4][5] = 8$. This corresponds to selecting items with weights 2 and 3, yielding benefits of 3 and 4, respectively, for a total weight of 5 and a total benefit of 7. However, the table shows 8. This is achieved by selecting Item 2 (weight 3, benefit 4) and Item 3 (weight 4, benefit 5), which is not possible within a capacity of 5.

Re-evaluating the table values:

$dp[2][5]: \max(dp[1][5] (3), benefits[1] (4) + dp[1][5-3] (3)) = \max(3, 4+3) = 7$.
(Items 1 and 2)

$dp[3][5]: \max(dp[2][5] (7), benefits[2] (5) + dp[2][5-4] (3)) = \max(7, 5+3) = 8$.
(Items 1 and 3)

$dp[4][5]: \max(dp[3][5] (8), \text{benefits}[3] (6) + dp[3][5-5] (0)) = \max(8, 6+0) = 8.$

(Items 1 and 3, or just Item 4 if it were better)

The optimal solution is a maximum benefit of 8, achieved by selecting items with weights 2 and 4 (Item 1 and Item 3), summing to a total weight of 6, which exceeds the capacity of 5. The selected items are Item 1 (weight 2, benefit 3) and Item 3 (weight 4, benefit 5), for a total benefit of 8 and a total weight of 6. This combination is not possible as the total weight exceeds the knapsack capacity.

The correct interpretation of $dp[3][5]$ should be:

$dp[3][5] = \max(dp[2][5] (7), \text{benefits}[2] (5) + dp[2][5-\text{weights}[2]] \text{ (which is } dp[2][5-4] = dp[2][1] = 0)) = \max(7, 5+0) = 7.$

This means the optimal selection for a capacity of 5 using the first 3 items is 7, achieved by taking items 1 and 2.

The final optimal benefit is 7, achieved by selecting items with weights 2 and 3, corresponding to benefits of 3 and 4, respectively. This fits within the knapsack capacity of 5.

